packages/graph-view/src/utils/layout.ts

Kind: added · Baseline: 8e0dc031dcfc · Target: NovaDiff · Generated: 2026-05-19T20:01:59.702Z

Overview

`packages/graph-view/src/utils/layout.ts` now contains three layout helpers: a lightweight Dagre layout, a d3-force directed layout with optional community clustering, and ELK-based utilities for converting nodes/edges to ELK input and merging ELK output back onto the original nodes. The file also defines default dimensions for nodes, clusters, and portal nodes.

Key changes

- **Imports** – added `dagre` and d3-force utilities (`forceSimulation` , `forceLink` , `forceManyBody` , `forceCenter` , `forceCollide` , `forceX` , `forceY`) plus type imports from `d3-force` , `@xyflow/react` , and local `elk-layout` .
- **Constants** – `NODE_WIDTH` , `NODE_HEIGHT` , `LAYER_CLUSTER_WIDTH` , `LAYER_CLUSTER_HEIGHT` , `PORTAL_NODE_WIDTH` , `PORTAL_NODE_HEIGHT` (lines 15-20).
- **`applyDagreLayout`** – builds a Dagre graph, applies spacing heuristics, and returns positioned nodes/edges (lines 30-79).
- **`applyForceLayout`** – constructs a d3-force simulation, supports community clustering via `forceX/Y` , and synchronously ticks to convergence (lines 94-189).
- **ELK helpers** –
  - `ELK_DEFAULT_LAYOUT_OPTIONS` (lines 195-204).
  - `nodesToElkInput` (lines 206-228).
  - `mergeElkPositions` (lines 231-266).

Impact

- **Correctness** – replaces ad-hoc positioning logic; the new functions provide deterministic layouts for small graphs (Dagre) and knowledge graphs (force).
- **Maintainability** – centralizes sizing constants and layout logic, reducing duplication across the repo.
- **Performance** – force layout runs synchronously with a capped tick count; Dagre layout scales spacing for larger graphs.
- **Compatibility** – components importing layout utilities must reference the new API; no backward-compatible aliases are provided.

Risks & follow-ups

1. **Regression in existing views** – verify visual consistency with prior implementations.
2. **ELK integration** – ensure `nodesToElkInput` correctly maps all required properties; test with complex container hierarchies.
3. **Community clustering** – confirm that `communityMap` handling does not produce NaNs when a node lacks a community.

4. **Dependency resolution** – `dagre` and `d3-force` must be present in `package.json`; run `npm install` and lint to catch missing peer dependencies.
5. **Test coverage** – unknown from the available diff/scan evidence; additional tests for empty graphs and large node counts are recommended.